Mario's Pizzabar

Systemudvikling

Gruppe 7

Celine (celineewu) | Eugenia (JennyEugenia)

Morten (mtothejensen) | Sebastian (SebastianH80)

Functional & Functional requirements:

Functional

ID	Requirement
FR1	Systemet skal kunne vise pizzamenuen
FR2	Systemet skal kunne oprette en ny ordre
FR3	Systemet skal kunne tilføje pizzaer til en ordre.
FR4	Systemet skal kunne registrere forskellige kundetyper
FR5	Systemet skal kunne beregne rabat baseret på kundetype
FR6	Systemet skal kunne vise en liste over aktive ordrer
FR7	Systemet skal kunne sortere ordrer efter afhentningstid
FR8	Systemet skal kunne markere en pizza som klar
FR9	Systemet skal kunne fjerne en ordre når den er afhentet
FR10	Systemet skal kunne gemme afsluttede ordrer i en fil
FR11	Systemet skal kunne indlæse gemte ordrer

Non-functional

ID	Requirement
NFR1	Systemet skal kunne køre på en ældre laptop
NFR2	Systemet skal kunne fungere uden internet
NFR3	Systemet skal være simpelt at bruge
NFR4	Systemet skal håndtere fejl uden at crashe
NFR5	Systemet skal validere brugerinput
NFR6	Systemet skal gemme data i filer
NFR7	Systemet skal kunne håndtere mindst 14 pizzaer
NFR8	Systemet skal være skrevet i Java

User Stories med Acceptkriterier (how to test)

US1. Som <brugere> vil jeg gerne kunne <se alle bestillinger>, så det kan give <et bedre overblik>

Accept kriteria:

1. Brugeren kan se alle sine bestillinger.
2. Hver bestilling viser dato, status og pris.
3. Bestillinger vises med nyeste først.

Test:
Vi gå ind i systmet og prøve at se om listen vises. vi tjekker at info (dato, status, pris) er korrekt og at rækkefølgen er nyeste først.

US2. Som <brugere> vil jeg gerne <opdatere bestillingslisten> så det bliver nemt at se <de uafhentede bestillinger*>

Accept kriterier:

1. Brugeren kan opdatere listen.
2. Listen viser os de nyeste data.

Test:
Vi gå ind i "bestilinger "og prøve at opdatere listen og se om det virker.

US 3(afhængig af US2). Som < brugere> vil jeg kunne oprette hurtig en ny bestilling så pizzaerne kan blive lavet på kort tid.

Accept kriterier:

1. Man, skal kunne se og vælge straks en kundetype, pizza og angive afhentningstid.
2. Ordren gemmes automatisk i systemet, og som resultat ordren vises i bestillingslisten.

Test:
Et ordre oprettes. Vi tjekker hvis listen indeholder korrekt pizza og afhentningstid.

US4. Som <brugere> vil jeg kunne se pizzamenuen på skærmen så jeg kan finde med det samme pizzaer når kunder bestiller.

Accept kriterier:

1. Menuen viser alle pizzaer.

2. Hver item viser navn og pris.
3. Menuen kan bruges omgående når kunderne bestiler og medarbejderne opretter ordre.

Test: Vi få adgang til menuen og tjekker om den indeholder alle pizzaer. Vi observere om den vises straks på skærmen og prøve at lave en ordre.

US5. Som <brugere> vil jeg kunne se bestillinger sorteret efter afhentningstid så jeg kan give besked videre til køken om hvilken pizza skal laves først.

Accept kriterier:

1. Ordrelisten sorteres automatisk efter tid.
2. Den tidligste ordre vises først.
3. Der vises navn, pizzatype og tid.

Test: Vi prøver at oprette et par bestilinger og tjekker om alle kriterier er opfyldt.

US6. Som <brugere> vil jeg kunne fjerne en ordre fra listen når pizzaen er afhentet og som resultat vises kun de aktive bestillinger.

Accept kriterier:

1. Et færdig rdre kan markeres som klar.
2. Når den er afhentet, fjernes automatisk fra listen.
3. Ordren gemmes i historik.

Test: Vi tilføj ordre, markerer som “afhentet”, bagefter tjekker om den ikke længere er i listen.

US7. Som <brugere> vil jeg gemme absolut alle færdige ordrer, så jeg kan på en senere tidspunkt se omsætning og statistik.

Accept kriterier:

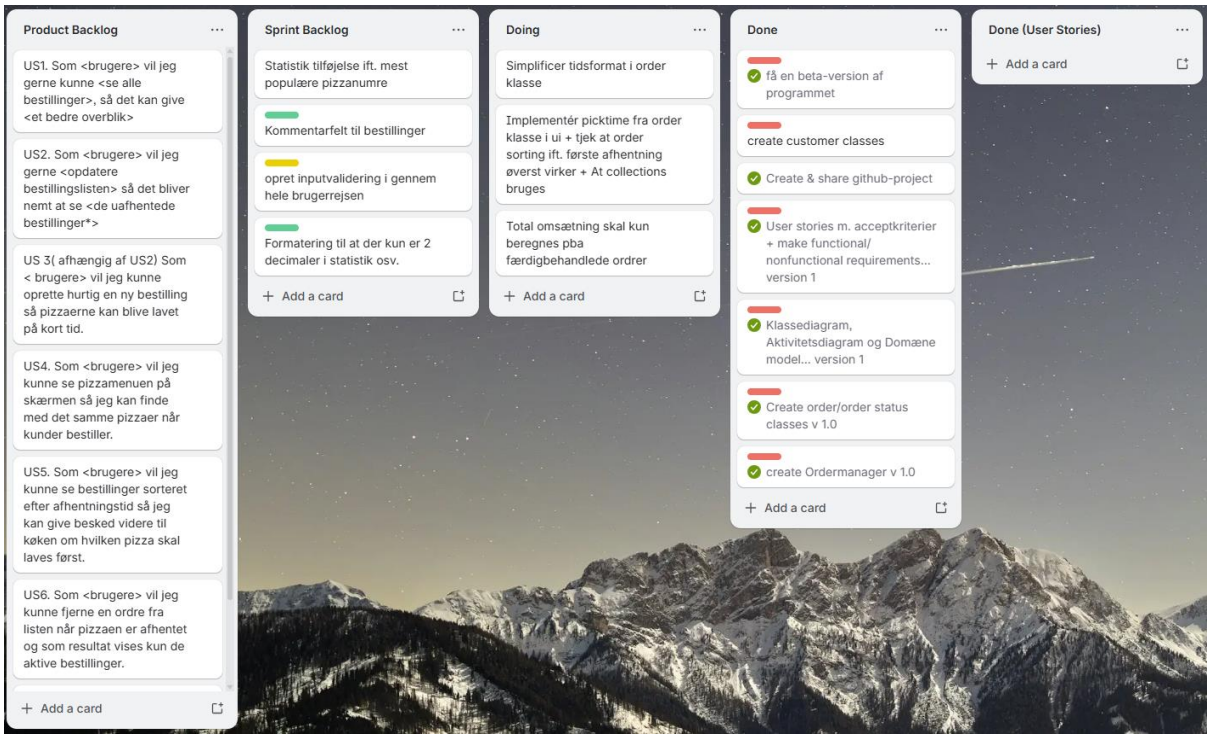
1. Man, kan se tidligere ordrer, som automatisk gemmes i historik.
2. Historikken må ikke slettes når programmet lukkes.
3. Historiken gemmes op til 5 år.

Test: Vi afslutter en ordre og tjekker at den gemmes i historik. Som sidste step vi genstarter programet.

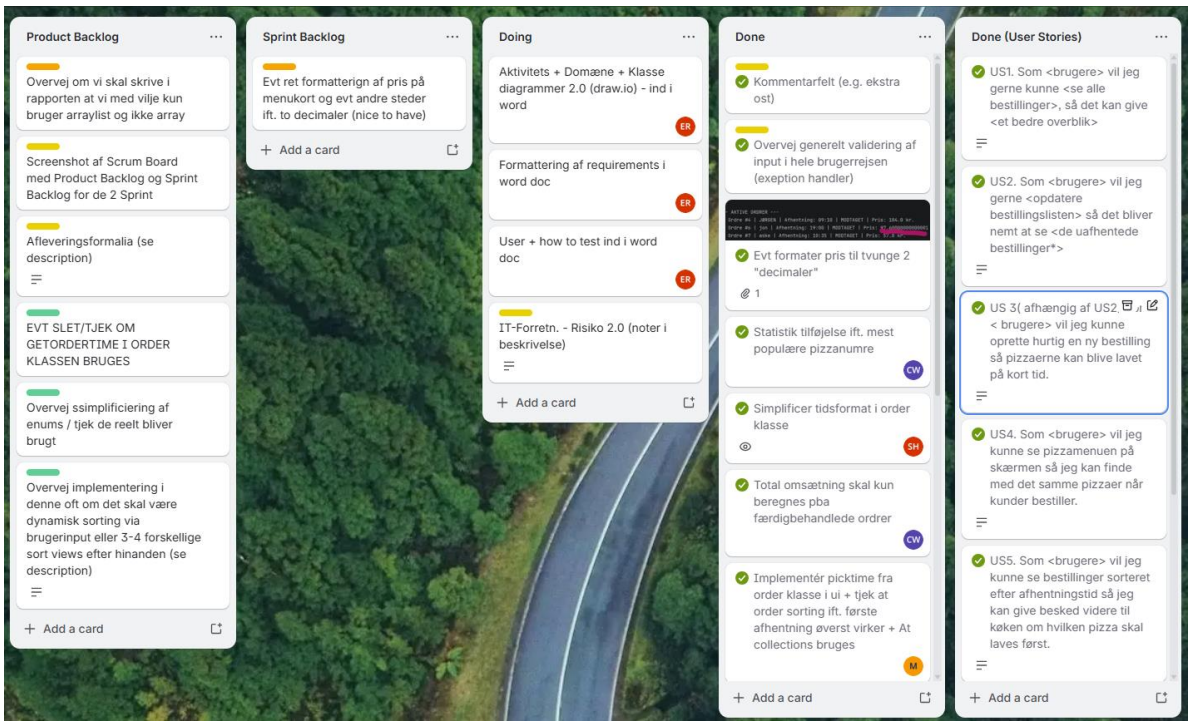
Scrum Boards

Screenshot af Scrum Board med Product Backlog og Sprint Backlog for de 2 Sprint

UGE 1:



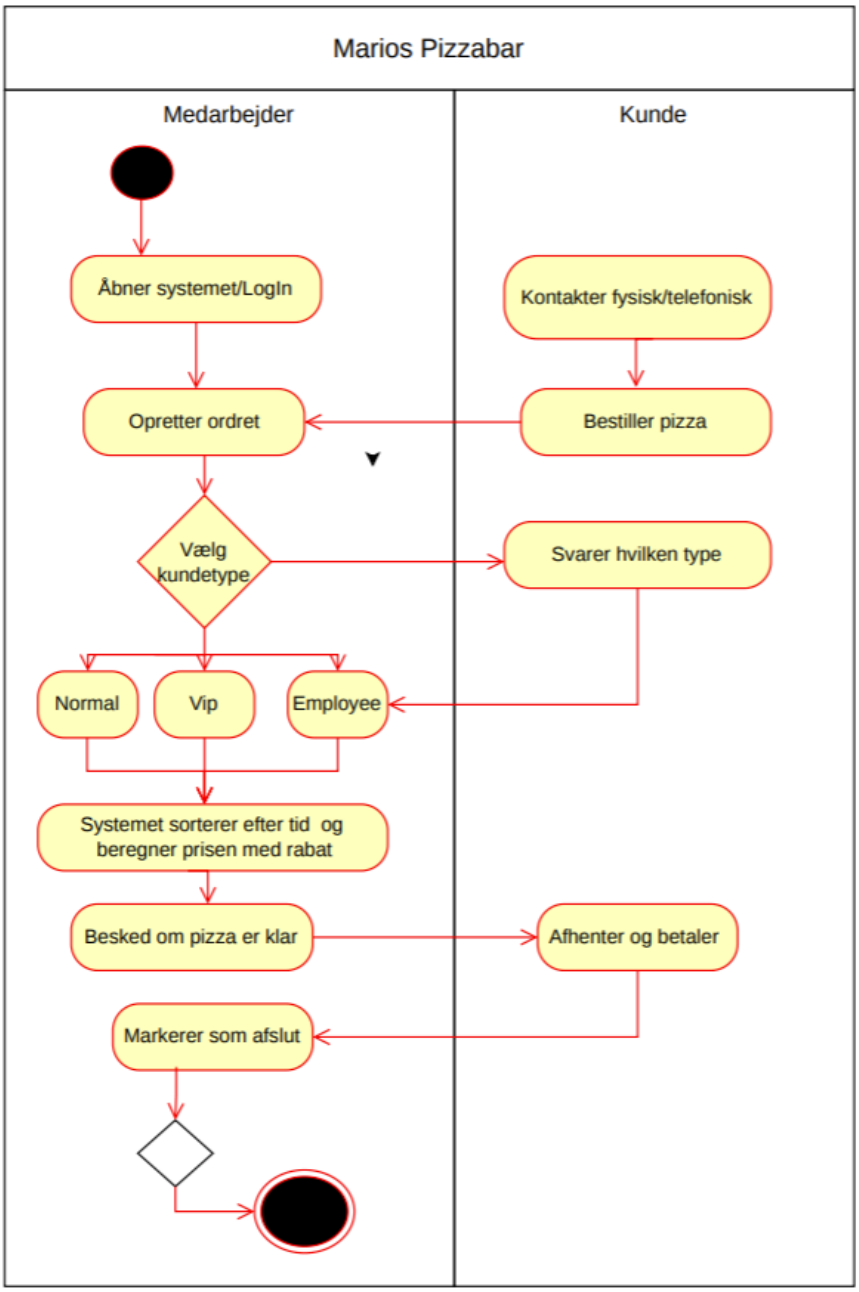
UGE 2:



Aktivitetsdiagram

Aktivitetsdiagram over brugerens interaktion med systemet.

Vores aktivitetsdiagram viser processen i systemet trin for trin og hvordan handlinger hænger sammen. Formålet er at vise hvordan vores proces fungerer fra start til slut.

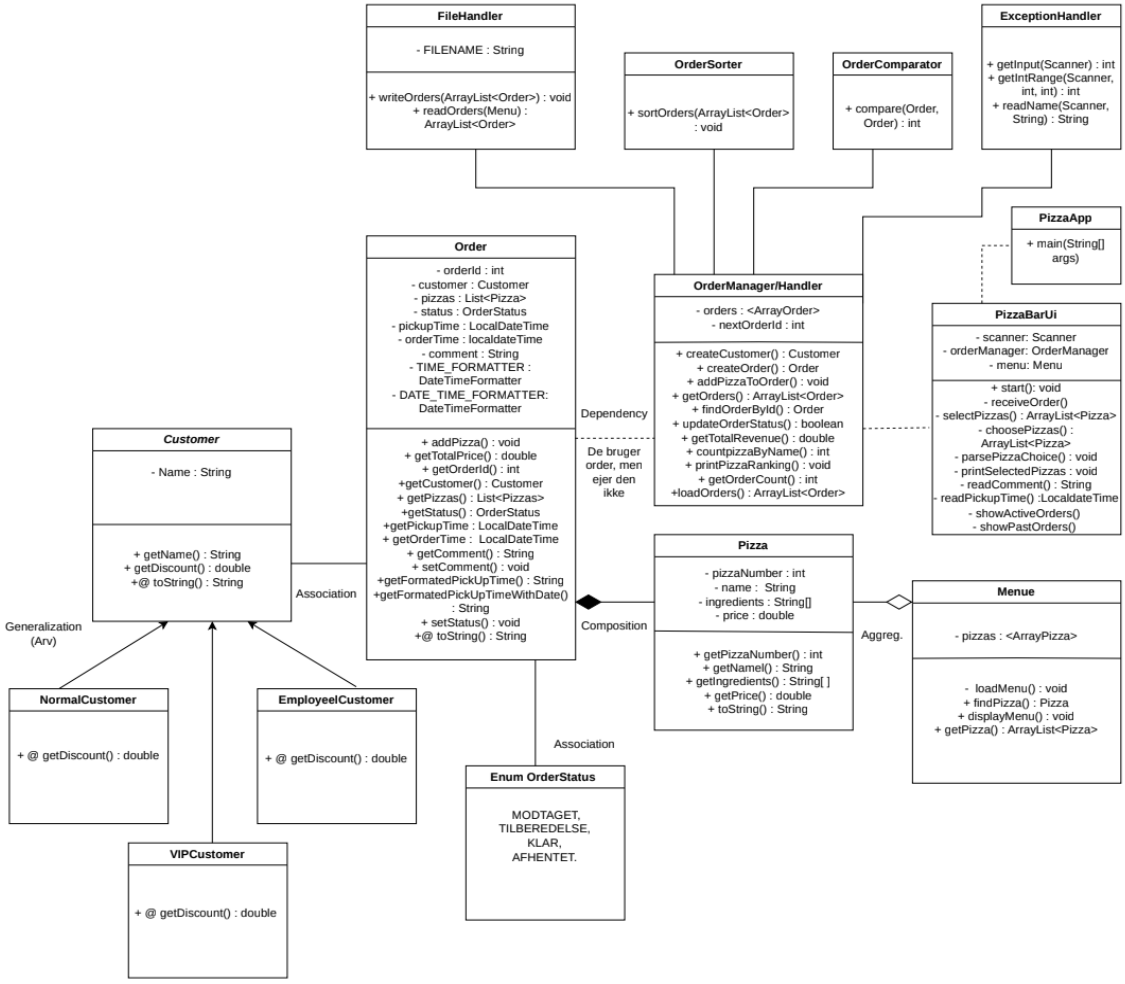


Klassediagram

Klassediagram over det færdige system

Vores klassediagram viser programmets opbygning med klasser, data og relationer. Det bruges til at forstå og designe systemet.

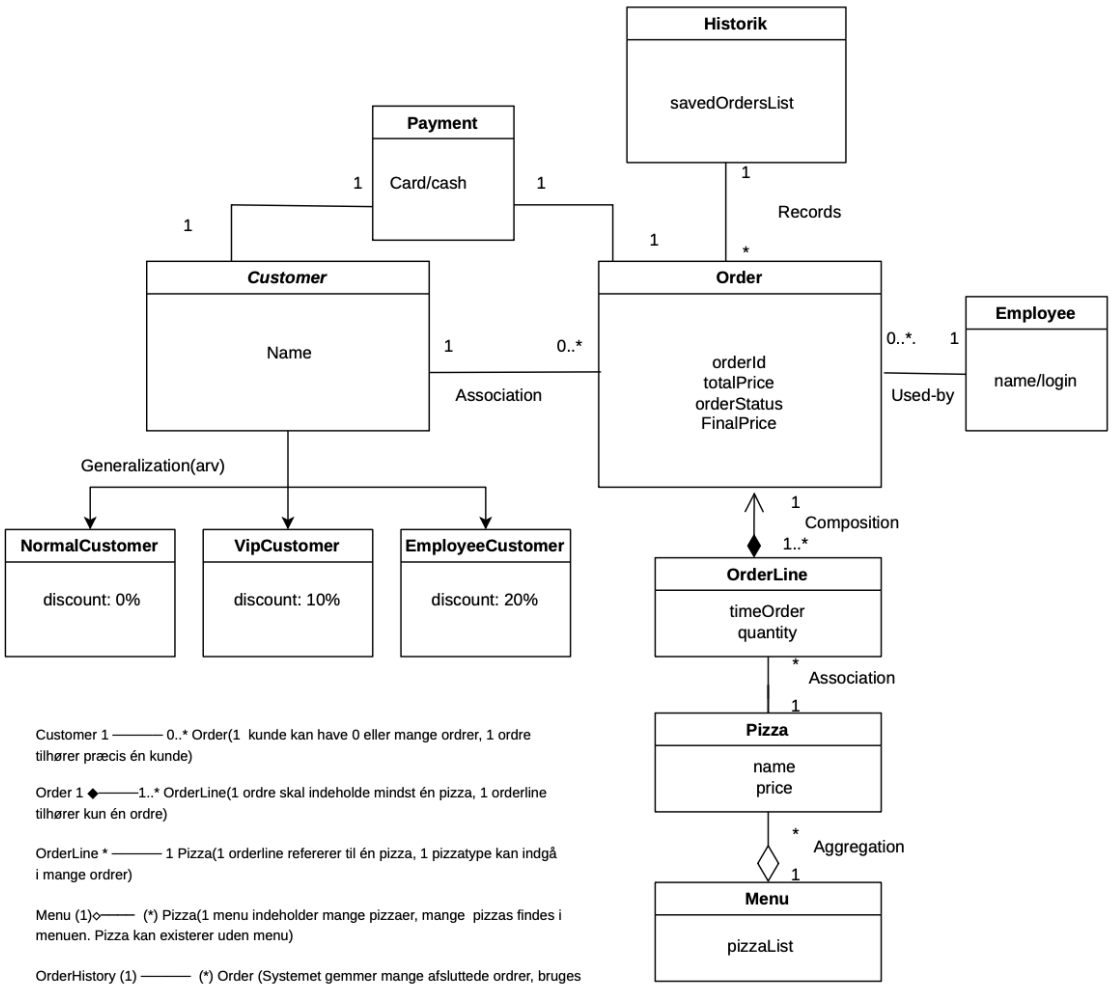
- Klasser (fx Order, Pizza, Customer)
- Attributter (fx navn, pris)
- Metoder (fx getPrice())
- Relationer:
    - arv (inheritance)
    - association (forbindelser)
    - Composition/aggregation


FileHandler
- FILENAME : String
+ writeOrders(ArrayList<Order>) : void
+ readOrders(Menu) : ArrayList<Order>
OrderSorter
+ sortOrders(ArrayList<Order> : void
OrderComparator
+ compare(Order, Order) : int
ExceptionHandler
+ getInput(Scanner) : int
+ getIntRange(Scanner, int, int) : int
+ readName(Scanner, String) : String
PizzaApp
+ main(String[] args)
Order
- orderId : int
- customer : Customer
- pizzas : List<Pizza>
- status : OrderStatus
- pickupTime : LocalDateTime
- orderTime : localdateTime
- comment : String
- TIME_FORMATTER : DateTimeFormatter
- DATE_TIME_FORMATTER: DateTimeFormatter
+ addPizza() : void
+ getTotalPrice() : double
+ getOrderId() : int
+getCustomer() : Customer
+ getPizzas() : List<Pizzas>
+getStatus() : OrderStatus
+getPickupTime : LocalDateTime
+ getOrderTime : LocalDateTime
+ getComment() : String
+ setComment() : void
+getFormatedPickUpTime() : String
+getFormatedPickUpTimeWithDate() : String
+ setStatus() : void
+@ toString() : String
OrderManager/Handler
- orders : <ArrayOrder>
- nextOrderId : int
+ createCustomer() : Customer
+ createOrder() : Order
+ addPizzaToOrder() : void
+ getOrders() : ArrayList<Order>
+ findOrderById() : Order
+ updateOrderStatus() : boolean
+ getTotalRevenue() : double
+ countpizzaByName() : int
+ printPizzaRanking() : void
+ getOrderCount() : int
+loadOrders() : ArrayList<Order>
PizzaBarUi
- scanner: Scanner
- orderManager: OrderManager
- menu: Menu
+ start(): void
- receiveOrder()
- selectPizzas() : ArrayList<Pizza>
- choosePizzas() : ArrayList<Pizza>
- parsePizzaChoice() : void
- printSelectedPizzas : void
- readComment() : String
- readPickupTime() :LocaldateTime
- showActiveOrders()
- showPastOrders()
Dependency
De bruger order, men ejer den ikke
Customer
- Name : String
+ getName() : String
+ getDiscount() : double
+@ toString() : String
Association
Generalization (Arv)
Composition
Pizza
- pizzaNumber : int
- name : String
- ingredients : String[]
- price : double
+ getPizzaNumber() : int
+ getNameI() : String
+ getIngredients() : String[ ]
+ getPrice() : double
+ toString() : String
Aggreg.
Menue
- pizzas : <ArrayPizza>
- loadMenu() : void
+ findPizza() : Pizza
+ displayMenu() : void
+ getPizza() : ArrayList<Pizza>
NormalCustomer
+ @ getDiscount() : double
EmployeelCustomer
+ @ getDiscount() : double
VIPCustomer
+ @ getDiscount() : double
Association
Enum OrderStatus
MODTAGET,
TILBEREDELSE,
KLAR,
AFHENTET.

Domæne Model

(konceptuel model)

Vores domænemodel viser de vigtigste begreber i PizzaSystemet og hvordan de hænger sammen. Vi kan tydeligt se de vigtigste objekter og deres relationer, uden detaljer som metoder. Derudover har vi noteret kardinalitet, som viser hvor mange objekter der er i en relation, fx én kunde til mange ordrer.



Customer 1 ——— 0..* Order(1 kunde kan have 0 eller mange ordrer, 1 ordre tilhører præcis én kunde)

Order 1 ◆——1..* OrderLine(1 ordre skal indeholde mindst én pizza, 1 orderline tilhører kun én ordre)

OrderLine * ——— 1 Pizza(1 orderline refererer til én pizza, 1 pizzatype kan indgå i mange ordrer)

Menu (1)◇—— (*) Pizza(1 menu indeholder mange pizzaer, mange pizzas findes i menuen. Pizza kan existerer uden menu)

OrderHistory (1) ——— (*) Order (Systemet gemmer mange afsluttede ordrer, bruges til statistik og omsætning)

Employee (1)——— (0...*) Order (1 medarbejder kan oprette mange ordrer, 1 ordre oprettes af én medarbejder)